CORRIGÉ · PROBLÈME · ANNALES SPÉ

Sous-graphe le plus dense : l'algorithme de Goldberg

Énoncé

Graphe non orienté $G = (V, E)$ à sommets $1, \dots, n$, listes d'adjacence doublement chaînées : `struct link_s { int node; struct link_s *prev, *next; }` ; `struct graph_s { unsigned int n; link **neighbours; }`, tableau de longueur $n+1$ (case 0 inutilisée) ; chaque arête est stockée comme deux arcs. Densité $\delta(G) = \frac{m}{n}$; $\rho(G) = \max_{X \neq \emptyset} \delta(G_X)$, où G_X est le sous-graphe induit par X.

1. `graph *gr_create(unsigned int n)` : graphe vide à n sommets.
2. `void gr_add(graph *G, int u, int v)` : insère l'arête $\{u, v\}$.
3. `int degree(graph *G, int v)`.
4. `int edges(graph *G)` : nombre d'arêtes.
5. `double density(graph *G)`.

Réseau associé à G (n sommets, m arêtes) et à un entier r : $W = V \cup \{s, t\}$; arc (s, v) de multiplicité mn^2 pour tout v ; arcs (u, v) et (v, u) de multiplicité n^2 pour toute arête ; arc (u, t) de multiplicité $mn^2 - n^2 \deg_G(u) + 2r$ pour tout u. Question : a-t-on $\rho(G) > \frac{r}{n^2}$?

1. `void reduce(graph *G, int r)` : place dans `H` le réseau associé.
2. Montrer $\delta(G_X) = \dfrac{\sum_{v \in X} \deg_G(v) - \operatorname{card}\{\{u, v\} \in E : u \in X,\ v \notin X\}}{2|X|}$.
3. Pour $S = \{s\} \cup X$: $\mu(\partial S) = mn^3 + 2|X|(r - n^2\delta(G_X))$, et $\mu(\partial\{s\}) = mn^3$.
4. Montrer : $\sigma(H) = mn^3 \iff \rho(G) \leqslant \frac{r}{n^2}$ et $\sigma(H) < mn^3 \iff \rho(G) > \frac{r}{n^2}$.
5. `bool oracle(graph *G, int r)` : renvoie vrai si et seulement si $\rho(G) > \frac{r}{n^2}$. Justifier.

$\Delta = \{(m', n') : 0 \leqslant m' \leqslant m,\ 1 \leqslant n' \leqslant n\}$.

1. Si $\frac{m_1}{n_1} \neq \frac{m_2}{n_2}$ (couples de Δ), alors $\left|\frac{m_1}{n_1} - \frac{m_2}{n_2}\right| \geqslant \frac{1}{n^2}$.
2. S'il n'existe aucun sous-graphe induit de densité $\geqslant \delta(G_X) + \frac{1}{n^2}$, alors $\delta(G_X) = \rho(G)$.
3. `void binary_search(graph *G)` : modifie `A` pour que $X = \{v \in V : A[v] \geqslant 0\}$ vérifie $\delta(G_X) = \rho(G)$. Justifier.

4. Comment faire interagir `densest_subgraph.c` avec `network.h` et `network.c` ?

Décoder l'énoncé

Ce que l'énoncé te donne.

- Dans le réseau associé, la source est 0, les sommets du graphe $1, \dots, n$, le puits $n+1$: la numérotation colle avec celle du réseau.

Ce qu'on te demande. Décider $\rho(G) > \frac{r}{n^2}$ par une coupe minimale, puis trouver $\rho(G)$ exactement par dichotomie sur r.

Corrigé

Question 17

Étape 1. Q17. gr_create

```
graph *gr_create(unsigned int n) {
    graph *G = malloc(sizeof(graph));
    G->n = n;
    G->neighbours = malloc((n + 1) * sizeof(link *));
    for (unsigned int v = 0; v <= n; v++)
        G->neighbours[v] = NULL;      /* liste d'adjacence vide */
    return G;
}
```

Question 18

Étape 1. Q18. gr_add

```
/* insère v en tête de la liste d'adjacence de u */
static void push_link(graph *G, int u, int v) {
    link *l = malloc(sizeof(link));
    l->node = v;
    l->prev = NULL;
    l->next = G->neighbours[u];
    if (l->next != NULL) l->next->prev = l;
    G->neighbours[u] = l;
}

void gr_add(graph *G, int u, int v) {
    push_link(G, u, v);    /* arc u -> v */
    push_link(G, v, u);    /* arc v -> u : l'arête est stockée deux fois */
}
```

Insertion en tête en $O(1)$, sans oublier le chaînage arrière (`prev`).

Question 19

Étape 1. Q19. degree

```
int degree(graph *G, int v) {
    int d = 0;
    for (link *l = G->neighbours[v]; l != NULL; l = l->next)
        d++;
    return d;
}
```

Question 20

Étape 1. Q20. edges

```
int edges(graph *G) {
    int somme = 0;
    for (unsigned int v = 1; v <= G->n; v++)
        somme += degree(G, v);
    return somme / 2;      /* chaque arête est stockée comme deux arcs */
}
```

Question 21

Étape 1. Q21. density

```
double density(graph *G) {
    return (double) edges(G) / G->n;
}
```

La conversion en `double` évite la division entière.

Question 22

Étape 1. Q22. Le réseau associé

```
void reduce(graph *G, int r) {
    unsigned int n = G->n;
    unsigned int m = edges(G);
    nw_init(n);                                  /* n + 2 sommets, aucun arc */
    for (unsigned int u = 1; u <= n; u++) {
        H.mu[0][u] = m * n * n;                  /* arc (s, u) */
        for (link *l = G->neighbours[u]; l != NULL; l = l->next)
            H.mu[u][l->node] = n * n;            /* arc (u, v) pour l'arête {u,v} */
        H.mu[u][n + 1] = m * n * n - n * n * degree(G, u) + 2 * r;   /* arc (u, t) */
    }
}
```

Chaque arête $\{u, v\}$ apparaît dans la liste de u (donne l'arc (u, v)) et dans celle de v (donne (v, u)). La multiplicité de (u, t) vaut $n^2(m - \deg u) + 2r \geqslant 0$; si elle est nulle, il n'y a simplement pas d'arc.

Question 23

Étape 1. Q23. Densité et degrés

Faisons la somme $\sum_{v \in X} \deg_G(v)$: chaque arête $\{u, v\}$ de G y compte autant de fois qu'elle a d'extrémités dans X :

- deux fois si $u, v \in X$ (arête de E_X) ;
- une fois si une seule extrémité est dans X (arête « coupée ») ;
- zéro fois sinon.

$$\sum_{v \in X} \deg_G(v) = 2|E_X| + c_X, \qquad c_X = \operatorname{card}\{\{u, v\} \in E : u \in X,\ v \notin X\}.$$

On isole $|E_X|$ et on divise par $|X|$:

$$\delta(G_X) = \frac{|E_X|}{|X|} = \frac{\sum_{v \in X} \deg_G(v) - c_X}{2|X|}.$$

Question 24

Étape 1. Q24. Bordure de $\{\}\{s\{\}\}\{\}cupX$

Les arcs qui sortent de $S = \{s\} \cup X$ sont :

- (s, v) pour $v \notin X$: $(n - |X|)$ arcs de multiplicité mn^2 ;

- (u,t) pour $u \in X$: multiplicité $mn^2 - n^2 \deg_G(u) + 2r$ chacun ;
- (u,v) avec $u \in X$, $v \notin X$, $\{u,v\} \in E$: c_X arcs de multiplicité n^2.

$$\mu(\partial S) = (n-|X|)mn^2 + |X|mn^2 - n^2 \sum_{u\in X} \deg_G(u) + 2r|X| + n^2 c_X = mn^3 + 2r|X| - n^2\Big(\sum_{u\in X} \deg_G(u) - c_X\Big).$$

Avec Q23, la parenthèse vaut $2|X|\delta(G_X)$:

$$\mu(\partial S) = mn^3 + 2|X|(r - n^2\delta(G_X)).$$

Pour $S = \{s\}$: seuls les n arcs (s,v) sortent, d'où $\mu(\partial\{s\}) = n \cdot mn^2 = mn^3$.

Question 25

Étape 1. Q25. Le test par la coupe minimale

Un entourage contient s, pas t : c'est $\{s\} \cup X$ pour un $X \subseteq V$. Par Q24, sa bordure vaut mn^3 si $X = \emptyset$, et $mn^3 + 2|X|(r - n^2\delta(G_X))$ sinon. Donc

$$\sigma(H) = \min\Big(mn^3,\ \min_{X\neq\emptyset}\big[mn^3 + 2|X|(r - n^2\delta(G_X))\big]\Big) \leqslant mn^3.$$

- $\sigma(H) < mn^3 \iff$ il existe $X \neq \emptyset$ avec $r - n^2\delta(G_X) < 0$, c'est-à-dire $\delta(G_X) > \frac{r}{n^2} \iff \rho(G) > \frac{r}{n^2}$.
- Comme toujours $\sigma(H) \leqslant mn^3$, la négation donne $\sigma(H) = mn^3 \iff \rho(G) \leqslant \frac{r}{n^2}$.

Question 26

Étape 1. Q26. L'oracle

```
bool oracle(graph *G, int r) {
    reduce(G, r);
    nw_disconnect();                          /* A : sommets accessibles = S0 */
    for (unsigned int v = 1; v <= G->n; v++)
        if (A[v] >= 0) return true;           /* S0 contient un sommet du graphe */
    return false;                             /* S0 = {s} */
}
```

Principe : après `nw_disconnect`, $S_0 = \{v : A[v] \geqslant 0\} = \{s\} \cup X_0$ est un entourage de plus petite bordure (Q16).

- Si $\rho(G) > \frac{r}{n^2}$: $\sigma(H) < mn^3$ (Q25), donc $X_0 \neq \emptyset$ (sinon la bordure vaudrait mn^3) : on renvoie `true`.
- Si $\rho(G) \leqslant \frac{r}{n^2}$: $\{s\}$ est lui-même de plus petite bordure. Or S_0 est inclus dans **tout** entourage minimal S : pour un tel S, l'argument de Q15 montre que les arcs qui sortent de S sont saturés

et que ceux qui y entrent ne sont pas utilisés, donc aucun arc du résiduel ne sort de S, et les sommets accessibles depuis s restent dans S. Avec $S = \{s\}$: $S_0 = \{s\}$, on renvoie `false`.

Bonus : quand la réponse est `true`, la ligne $\mu(\partial S_0) < mn^3$ de Q24 montre que $\delta(G_{X_0}) > \frac{r}{n^2}$: `A` désigne un sous-graphe témoin.

Question 27

Étape 1. Q27. Deux densités distinctes sont éloignées

$$\left|\frac{m_1}{n_1} - \frac{m_2}{n_2}\right| = \frac{|m_1 n_2 - m_2 n_1|}{n_1 n_2}.$$

Le numérateur est un entier, non nul puisque les fractions diffèrent : il vaut au moins 1. Le dénominateur vérifie $n_1 n_2 \leqslant n^2$. D'où

$$\left|\frac{m_1}{n_1} - \frac{m_2}{n_2}\right| \geqslant \frac{1}{n^2}.$$

Question 28

Étape 1. Q28. Un critère d'optimalité

Soit Y réalisant $\rho(G) = \delta(G_Y)$. Par définition, $\rho(G) \geqslant \delta(G_X)$. Les deux densités sont de la forme $\frac{m'}{n'}$ avec $(m', n') \in \Delta$ (un sous-graphe induit a au plus m arêtes et entre 1 et n sommets).

Si $\rho(G) \neq \delta(G_X)$, Q27 donne $\rho(G) \geqslant \delta(G_X) + \frac{1}{n^2}$: G_Y serait un sous-graphe induit de densité $\geqslant \delta(G_X) + \frac{1}{n^2}$, ce que l'hypothèse exclut. Donc $\delta(G_X) = \rho(G)$.

Question 29

Étape 1. Q29. Recherche dichotomique

Encadrement : $0 \leqslant \rho(G) \leqslant \frac{n-1}{2}$ (un sommet a au plus $n-1$ voisins, donc $m' \leqslant \frac{n'(n'-1)}{2}$). Avec $r_{\max} = \frac{n^2(n-1)}{2}$ (entier), l'oracle répond `false` en $r_{\max}$. La réponse de l'oracle est décroissante en r (vraie pour $r < n^2\rho$, fausse ensuite).

```
void binary_search(graph *G) {
    int n = G->n;
    int lo = -1;                      /* oracle(lo) vrai (r = -1 : convention) */
    int hi = n * n * (n - 1) / 2;     /* oracle(hi) faux */
    while (hi - lo > 1) {
        int mid = (lo + hi) / 2;
        if (oracle(G, mid)) lo = mid; else hi = mid;
    }
    if (lo == -1) {                   /* rho(G) = 0 : pas d'arête */
        _arr_init(A); A[0] = 0; A[1] = 0;      /* un sommet seul convient */
    } else
        oracle(G, lo);                /* A décrit X avec delta(G_X) > lo / n^2 */
}
```

Correction : à la fin, $hi = lo + 1$, $\rho(G) > \frac{lo}{n^2}$ et $\rho(G) \leqslant \frac{lo+1}{n^2}$. Le dernier appel `oracle(G, lo)` laisse dans `A` un ensemble X avec $\delta(G_X) > \frac{lo}{n^2}$ (bonus de Q26). Si $\delta(G_X) \neq \rho(G)$, Q27 donnerait $\delta(G_X) \leqslant \rho(G) - \frac{1}{n^2} \leqslant \frac{lo}{n^2}$: contradiction. Donc $\delta(G_X) = \rho(G)$.

Efficacité : $O(\log n^3) = O(\log n)$ appels à l'oracle, au lieu de tester toutes les valeurs de r.

Question 30

Étape 1. Q30. Compilation des trois fichiers

- `densest_subgraph.c` commence par `#include "network.h"` pour connaître le type `network` et les fonctions `nw_init`, `nw_disconnect`, `_arr_init`.
- Les variables globales `H` et `A` sont **définies** dans `network.c` ; pour les utiliser ailleurs, il faut les **déclarer** : `extern network H; extern int A[NMAX];` (idéalement ajoutées dans `network.h`).
- On compile chaque fichier en objet puis on lie : `gcc -c network.c`, `gcc -c densest_subgraph.c`, `gcc network.o densest_subgraph.o -o densest`.

Le piège

En Q26, répondre `true` dès que $\sigma(H) \leqslant mn^3$: il faut un X non vide, et c'est la minimalité de l'ensemble accessible S_0 qui garantit $S_0 = \{s\}$ dans le cas d'égalité.

La méthode

Réduction à une coupe minimale : construire un réseau dont la bordure de $\{s\} \cup X$ s'exprime avec la quantité à optimiser, puis dichotomie sur un seuil entier grâce à l'écart minimal entre fractions.

Rappels de cours utilisés

- **Compilation séparée en C.** Chaque fichier `.c` inclut les en-têtes `.h` dont il a besoin et se compile en un fichier objet (`gcc -c f.c`). L'édition de liens assemble les objets (`gcc f.o g.o -o prog`). Une variable globale définie dans un fichier s'utilise ailleurs grâce à une déclaration `extern`.